

JAVA PROGRAMMING MASTERCLASS

# Stream 與 Lambda

---

「用更少的程式碼，做更多的事」

[← 返回目錄](#)

# Outline

- 第一部分：Lambda 運算式
  - 語法形式、函數式介面
  - 常用內建介面：`Predicate`、`Function`、`Consumer`、`Supplier`
  - JDK 11 新增：Lambda 中的 `var` 參數
- 第二部分：方法參考 (Method Reference)
  - `::` 運算子的四種形式
- 第三部分：Stream API
  - 中間操作：`filter`、`map`、`flatMap`、`sorted` ...
  - Primitive Stream：`mapToInt`、`mapToDouble`
  - JDK 9 新增：`takeWhile`、`dropWhile`
  - 終端操作：`forEach`、`collect`、`count` ...
  - JDK 16 新增：簡潔的 `.toList()`
  - JDK 12 新增：`teeing` 收集器
  - `Collectors` 工具
  - 實作練習



# 第一部分

## Lambda 運算式

# 什麼是 Lambda ？

Lambda 是 Java 8 引入的匿名函式語法，讓你不需要宣告完整的類別或方法，直接將「行為」當作參數傳遞。

- 精簡程式碼 — 取代冗長的匿名類別寫法
- 搭配集合框架 — 與 `List.forEach()`、`Stream` 等方法無縫整合
- 核心概念 — let Java support functional programming style

💡 基本語法： `(參數列) → 運算式 或 { 程式碼區塊 }`



# Lambda 語法形式

| 語法形式  | 範例                                     | 說明                        |
|-------|----------------------------------------|---------------------------|
| 無參數   | <code>() → "Hello"</code>              | 小括號不可省略                   |
| 單一參數  | <code>x → x * 2</code>                 | 括號可省略                     |
| 多個參數  | <code>(a, b) → a + b</code>            | 多參數需加括號                   |
| 多行程式碼 | <code>(x) → { ... ; return x; }</code> | 需大括號與 <code>return</code> |

```
1 Runnable r = () → System.out.println("Hello!");
2 Consumer<Integer> dbl = x → System.out.println(x * 2);
3 Comparator<Integer> cmp = (a, b) → a - b;
```

# Lambda 語法 – 比較傳統寫法

```
1  List<String> heroes = new ArrayList<>(
2      List.of("炭治郎", "禰豆子", "善逸", "伊之助"));
3
4  // ✗ 傳統匿名類別寫法 (冗長)
5  Collections.sort(heroes, new Comparator<String>() {
6      public int compare(String a, String b) {
7          return a.compareTo(b);
8      }
9  });
10
11 // ✓ Lambda 寫法 (簡潔)
12 Collections.sort(heroes, (a, b) → a.compareTo(b));
```



# 函數式介面 (Functional Interface)

Lambda 實際上是函數式介面的匿名實作。函數式介面只含一個抽象方法。

```
1  @FunctionalInterface
2  interface Greeting {
3      String greet(String name);
4  }
5
6  // Lambda 直接實作介面
7  Greeting g = name → "哈囉，" + name;
8  System.out.println(g.greet("炭治郎")); // 哈囉，炭治郎
```



**@FunctionalInterface**

是可選的注解，但加上後若介面超過一個抽象方法，編譯器會立即報錯

# 常用內建函數式介面

| 介面                             | 方法簽名                | 說明                 |
|--------------------------------|---------------------|--------------------|
| <b>Predicate&lt;T&gt;</b>      | <b>T → boolean</b>  | 判斷條件，回傳 true/false |
| <b>Function&lt;T, R&gt;</b>    | <b>T → R</b>        | 輸入一個值，轉換後回傳        |
| <b>Consumer&lt;T&gt;</b>       | <b>T → void</b>     | 接受一個值，執行操作（無回傳）    |
| <b>Supplier&lt;T&gt;</b>       | <b>() → T</b>       | 不接受參數，提供一個值        |
| <b>Comparator&lt;T&gt;</b>     | <b>(T, T) → int</b> | 比較兩個物件的大小          |
| <b>BiFunction&lt;T,U,R&gt;</b> | <b>(T, U) → R</b>   | 接受兩個參數，回傳結果        |



# 常用內建函數式介面 – 範例

```
1  Predicate<Integer> isAdult = age → age ≥ 18;
2  System.out.println(isAdult.test(20)); // true
3  System.out.println(isAdult.test(15)); // false
4
5  Function<String, Integer> len = s → s.length();
6  System.out.println(len.apply("炭治郎")); // 3
7
8  Consumer<String> print = s → System.out.println("★ " + s);
9  print.accept("鬼殺隊"); // ★ 鬼殺隊
10
11 Supplier<String> title = () → "無限列車";
12 System.out.println(title.get()); // 無限列車
```

# Lambda 中的 var 參數 (JDK 11)

自 Java 11 起，Lambda 參數可以使用 `var` 關鍵字，這讓語法更一致：

範例

說明

`(var x, var y) → x + y`

所有參數都必須使用 `var`

`(@NonNull var x) → ...`

主要用途：方便在參數上加上註解 (Annotation)

```
1 // 傳統寫法
2 (String s) → s.toLowerCase()
3
4 // JDK 11 使用 var
5 (var s) → s.toLowerCase()
6
7 // 搭配註解 (必須使用類型或 var)
8 (@NonNull var s) → s.toLowerCase()
```

💡 規則：不能混用，例如 `(var x, y) → ...` 是不允許的。



**第二部分**

**方法參考**

**Method Reference**

# 方法參考語法

**::** 運算子讓你直接引用現有的方法，讓程式碼更簡潔：

| 語法                                       | 說明      | 等同 Lambda                            |
|------------------------------------------|---------|--------------------------------------|
| <code>ClassName :: staticMethod</code>   | 靜態方法    | <code>x → ClassName.method(x)</code> |
| <code>obj :: instanceMethod</code>       | 特定物件的方法 | <code>x → obj.method(x)</code>       |
| <code>ClassName :: instanceMethod</code> | 任意物件的方法 | <code>x → x.method()</code>          |
| <code>ClassName :: new</code>            | 建構子     | <code>x → new ClassName(x)</code>    |

```
1 List<String> heroes = List.of("炭治郎", "善逸", "伊之助");
2 heroes.forEach(System.out :: println);           // obj :: instanceMethod
3 heroes.stream().map(String :: length);           // ClassName :: instanceMethod
```



# 方法參考 – 四種形式範例

```
1 // 1. ClassName::staticMethod
2 List.of("1","2").stream().map(Integer::parseInt);
3 // 2. obj::instanceMethod
4 List.of("炭治郎","善逸").forEach(System.out::println);
5 // 3. ClassName::instanceMethod
6 List.of("炭","治郎").stream().map(String::length);
7 // 4. ClassName::new
8 Stream.of(1,2,3).collect(Collectors.toCollection(ArrayList::new));
```

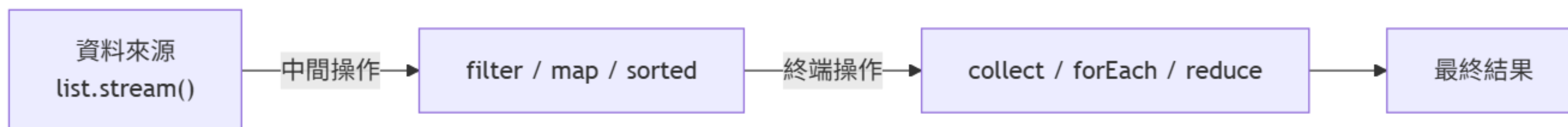


# 第三部分

## Stream API

# 什麼是 Stream ？

**Stream** 是 Java 8 引入的資料流處理管道，讓集合操作可以宣告式地串接。



💡 **延遲執行**：中間操作不會立即執行，直到遇到終端操作才統一觸發。Stream 也不修改原始集合。



# Stream 的建立方式

| 方式              | 語法                              | 說明        |
|-----------------|---------------------------------|-----------|
| 從 Collection 建立 | <code>list.stream()</code>      | 最常見的方式    |
| 從陣列建立           | <code>Arrays.stream(arr)</code> | 適用於基本型態陣列 |
| 直接建立            | <code>Stream.of(a, b, c)</code> | 直接指定元素    |

```
1 List<String> list = List.of("A", "B", "C");
2 Stream<String> s1 = list.stream();
3
4 int[] arr = {1, 2, 3};
5 IntStream s2 = Arrays.stream(arr);
```

# 中間操作 (Intermediate Operations)

中間操作回傳新的 **Stream**，可以串接多個：

| 方法                        | 說明                         |
|---------------------------|----------------------------|
| <b>filter(Predicate)</b>  | 篩選符合條件的元素                  |
| <b>map(Function)</b>      | 將每個元素轉換為另一種型態              |
| <b>sorted()</b>           | 依自然順序排序                    |
| <b>sorted(Comparator)</b> | 依自訂比較器排序                   |
| <b>distinct()</b>         | 移除重複元素 ( 依 <b>equals</b> ) |
| <b>limit(long n)</b>      | 取前 n 個元素                   |
| <b>skip(long n)</b>       | 跳過前 n 個元素                  |



# 中間操作 – 範例

```
1  List<Integer> nums = List.of(5, 3, 1, 4, 2, 3, 5);
2
3  nums.stream()
4      .filter(n → n > 2)    // [5, 3, 4, 3, 5]
5      .distinct()          // [5, 3, 4]
6      .sorted()            // [3, 4, 5]
7      .limit(2)            // [3, 4]
8      .forEach(System.out::println);
9  // 輸出：3
10 //      4
```



# flatMap – 攤平巢狀結構

將「集合中的集合」攤平為單一串流：

```
1  List<List<String>> nested = List.of(  
2      List.of("水柱", "炭治郎"),  
3      List.of("雷柱", "善逸")  
4  );  
5  List<String> flat = nested.stream()  
6      .flatMap(List::stream)  
7      .toList();  
8  System.out.println(flat); // [水柱, 炭治郎, 雷柱, 善逸]
```

# Primitive Stream (基本型態串流)

`map()` 回傳 `Stream<T>` ; 若需統計數值，改用 `mapToInt` 、 `mapToDouble` :

| 方法                                         | 回傳型態                      | 常用終端操作                                                                                |
|--------------------------------------------|---------------------------|---------------------------------------------------------------------------------------|
| <code>mapToInt(ToIntFunction)</code>       | <code>IntStream</code>    | <code>sum()</code> 、 <code>average()</code> 、 <code>min()</code> 、 <code>max()</code> |
| <code>mapToDouble(ToDoubleFunction)</code> | <code>DoubleStream</code> | 同上 ( 回傳 double )                                                                      |

```
1 List<String> names = List.of("炭治郎", "禰豆子", "善逸");
2 double avg = names.stream()
3     .mapToInt(String::length).average().getAsDouble();
4 System.out.println(avg); // 2.333 ...
```



# 有序串流的斷句處理 (JDK 9)

JDK 9 新增了兩個處理有序 ( Sorted ) 串流的強大工具：

| 方法                                | 說明                         |
|-----------------------------------|----------------------------|
| <code>takeWhile(Predicate)</code> | 從頭開始取，直到條件 <b>不成立</b> 就停止  |
| <code>dropWhile(Predicate)</code> | 從頭開始丟，直到條件 <b>不成立</b> 才開始取 |

```
1 List<Integer> nums = List.of(1, 2, 3, 4, 5, 4, 3, 2, 1);
2
3 // takeWhile: 取出 < 4 的元素 (遇到 4 即停止)
4 nums.stream().takeWhile(n → n < 4); // [1, 2, 3]
5
6 // dropWhile: 丟掉 < 4 的元素 (遇到 4 才開始取)
7 nums.stream().dropWhile(n → n < 4); // [4, 5, 4, 3, 2, 1]
```

💡 與 **filter** 的不同：filter 會檢查「所有」元素；takeWhile 只要條件一失敗就立刻收工。

# 終端操作 (Terminal Operations)

終端操作結束 **Stream** 管道，產生最終結果：

| 方法                                            | 說明                                 |
|-----------------------------------------------|------------------------------------|
| <code>forEach(Consumer)</code>                | 對每個元素執行操作（無回傳值）                    |
| <code>collect(Collector)</code>               | 收集結果到集合中                           |
| <code>reduce(identity, BinaryOperator)</code> | 將元素合併為單一值                          |
| <code>count()</code>                          | 回傳元素數量                             |
| <code>min(Comparator)</code>                  | 找最小值（回傳 <code>Optional</code> ）    |
| <code>max(Comparator)</code>                  | 找最大值（回傳 <code>Optional</code> ）    |
| <code>findFirst()</code>                      | 取得第一個元素（回傳 <code>Optional</code> ） |



# 終端操作 – 範例

```
1  List<Integer> scores = List.of(85, 72, 91, 60, 88, 95);
2
3  // count: 計算 ≥ 80 的人數
4  long cnt = scores.stream().filter(s → s ≥ 80).count(); // 4
5
6  // reduce: 加總所有分數
7  int sum = scores.stream().reduce(0, Integer::sum);      // 491
8
9  // collect: 篩選後收集
10 List<Integer> top = scores.stream()
11     .filter(s → s ≥ 80)
12     .sorted()
13     .collect(Collectors.toList());
14 System.out.println(top); // [85, 88, 91, 95]
```



# Optional – 安全的可能空值

`min()`、`max()`、`findFirst()` 回傳 `Optional<T>`，代表「可能有值、也可能沒有」：

| 方法                               | 說明                      |
|----------------------------------|-------------------------|
| <code>isPresent()</code>         | 有值時回傳 <code>true</code> |
| <code>get()</code>               | 取得值（無值時拋出異常）            |
| <code>orElse(T default)</code>   | 無值時回傳預設值                |
| <code>orElseGet(Supplier)</code> | 無值時呼叫 Supplier          |

```
1 Optional<Integer> max = List.of(3, 1, 5).stream()
2   .max(Comparator.naturalOrder());
3 System.out.println(max.isPresent()); // true
4 System.out.println(max.orElse(-1)); // 5
```

# Stream 轉 List 的捷徑 (JDK 16)

以往將 Stream 轉回 List 需要寫一段冗長的語法，JDK 16 提供了優雅的捷徑：

| 版本       | 語法                                         | 備註                            |
|----------|--------------------------------------------|-------------------------------|
| Java 8+  | <code>.collect(Collectors.toList())</code> | 回傳的可變性視實作而定                   |
| Java 16+ | <code>.toList()</code>                     | 回傳一個不可變 (Unmodifiable) 的 List |

```
1 List<String> list = Stream.of("A", "B", "C").toList();
2 // 等同於舊版的 .collect(Collectors.toList()) 但更簡潔！
```



# 技巧：快速反轉條件 (JDK 11)

使用 `Predicate.not()` 讓過濾邏輯讀起來更像英文，提升程式碼可讀性：

```
1  List<String> lines = List.of("A", " ", "B", "");
2
3  // 傳統寫法：過濾掉空白（邏輯較不明確）
4  lines.stream().filter(s → !s.isBlank());
5
6  // JDK 11 寫法：過濾「不是空白」的字串
7  lines.stream().filter(Predicate.not(String::isBlank));
```

💡 搭配方法參考：這種寫法特別適合與 `ClassName::methodName` 結合使用。

# Collectors 常用工具

方法

說明

`Collectors.toList()`

收集為 `List`

`Collectors.joining(delimiter)`

字串串接，可指定分隔符

`Collectors.groupingBy(Function)`

依條件分組，回傳 `Map`

```
1 List<String> names = List.of("炭治郎", "善逸", "伊之助");
2 String joined = names.stream()
3     .collect(Collectors.joining("、"));
4 System.out.println(joined); // 炭治郎、善逸、伊之助
5 Map<Integer, List<String>> byLen = names.stream()
6     .collect(Collectors.groupingBy(String::length));
```



# 雙向收集器：teeing (JDK 12)

**Collectors.teeing** 允許你將同一個串流分流給兩個收集器，最後再將結果合併：

```
1 // 範例：同時計算「及格人數」與「平均分數」
2 var result = Stream.of(85, 45, 90, 62)
3   .collect(Collectors.teeing(
4     Collectors.filtering(s → s ≥ 60, Collectors.count()), // 收集
器 1
5     Collectors.averagingInt(s → s),                        // 收
集器 2
6     (count, avg) → "及格人數：" + count + "，平均：" + avg // 合併
邏輯
7   ));
8 System.out.println(result); // 及格人數：3，平均：70.5
```

💡 設計目的：避免為了得到多個統計結果而對同一個集合進行多次 Stream 操作。



# Stream 完整應用範例

```
1  import java.util.*;
2  import java.util.stream.*;
3
4  List<Integer> scores = List.of(85, 72, 91, 60, 88, 95, 45);
5  // 取得所有及格 (≥60) 且高分 (≥80) 的成績，排序後輸出
6  List<Integer> result = scores.stream()
7      .filter(s → s ≥ 60)
8      .filter(s → s ≥ 80)
9      .sorted()
10     .collect(Collectors.toList());
11  System.out.println(result); // [85, 88, 91, 95]
12  int sum = scores.stream().reduce(0, Integer::sum);
13  System.out.println("總分：" + sum); // 536
```

# 練習一：篩選與串接英雄名單

## 任務說明

宣告一個 `List<String>` 包含以下名字：「炭治郎、禰豆子、善逸、伊之助、蜜璃、甘露寺、時透無一郎」

用 Stream 完成以下操作：

1. 篩選出名字長度  $\geq 3$  個字的人
2. 依字典順序排序
3. 用 `Collectors.joining(", ")` 串接後印出一行字串



# 練習一：解題提示

## 提示說明

1. `filter(name → name.length() ≥ 3)` 篩選
2. `sorted()` 依字典排序
3. `.collect(Collectors.joining("、"))` 串接為一行

```
1 List<String> heroes = List.of(  
2     "炭治郎", "禰豆子", "善逸", "伊之助", "蜜璃", "甘露寺", "時透無一郎");  
3 String result = heroes.stream()  
4     .filter(n → n.length() ≥ 3)  
5     .sorted()  
6     .collect(Collectors.joining("、"));  
7 System.out.println(result);
```

# 練習二：成績串流統計

## 任務說明

宣告 `List<Integer>` 成績：`{45, 78, 90, 62, 55, 85, 91, 73}`

用 Stream 完成以下操作：

1. 計算及格 ( $\geq 60$ ) 的人數
2. 找出所有成績中的最高分
3. 計算及格學生的平均分 (提示：使用 `mapToInt().average()`)



# 練習二：解題提示

## 提示說明

1. `filter( ... ).count()` 計算數量
2. `max(Comparator.naturalOrder()).getAsInt()` 取最大值
3. 先 filter 再 `mapToInt(Integer::intValue).average()`

```
1 List<Integer> s = List.of(45, 78, 90, 62, 55, 85, 91, 73);
2 long cnt = s.stream().filter(x → x ≥ 60).count();
3 int max = s.stream().max(Comparator.naturalOrder()).get();
4 double avg = s.stream().filter(x → x ≥ 60)
5     .mapToInt(Integer::intValue).average().getAsDouble();
6 System.out.printf("及格：%d 人，最高：%d，平均：%.1f%n", cnt, max, avg);
```



實戰加料

為期中作業暖身 🔥

# LocalDate – 日期的最佳拍檔

`java.time.LocalDate` 是 Java 8 引入的不可變日期物件，不含時區，用來處理「年月日」正好夠用：

| 方法                                          | 說明         |
|---------------------------------------------|------------|
| <code>LocalDate.now()</code>                | 取得今天的日期    |
| <code>LocalDate.of(year, month, day)</code> | 建立指定日期     |
| <code>date.isBefore(other)</code>           | 是否早於 other |
| <code>date.isAfter(other)</code>            | 是否晚於 other |
| <code>date.isEqual(other)</code>            | 是否同一天      |
| <code>date.plusDays(n)</code>               | 加 n 天後的日期  |

```
1 import java.time.LocalDate;  
2
```



# 實戰：LocalDate 判斷問卷狀態

問卷的「尚未開始 / 進行中 / 已結束」完全取決於今天與開始/結束日期的關係：

```
1  import java.time.LocalDate;
2
3  LocalDate today = LocalDate.now();
4  LocalDate startDate = LocalDate.of(2025, 6, 1);
5  LocalDate endDate   = LocalDate.of(2025, 6, 30);
6
7  String status;
8  if (today.isBefore(startDate)) {
9      status = "尚未開始"; // 停用問卷連結
10 } else if (today.isAfter(endDate)) {
11     status = "已結束";    // 停用問卷連結，但可看統計
12 } else {
13     status = "進行中";    // 啟用問卷連結
14 }
15 System.out.println("問卷狀態：" + status);
```

💡 動態問卷連結：前台列表頁「名稱的連結是否啟用」、後台列表頁的「可修改/可刪除/可觀看結果」條件，都是這個邏輯的延伸。

# 實戰：Stream + LocalDate 篩選問卷

學完 Stream 與 LocalDate，把兩者合體——一行篩出所有「進行中」的問卷：

```
1  import java.time.LocalDate;
2  import java.util.*;
3  import java.util.stream.*;
4
5  record Quiz(String name, LocalDate start, LocalDate end) {}
6
7  List<Quiz> quizzes = List.of(
8      new Quiz("滿意度調查", LocalDate.of(2025, 5, 1), LocalDate.of(2025,
9  5, 31)),
10     new Quiz("課程意見回饋", LocalDate.of(2025, 6, 1), LocalDate.of(202
11  5, 6, 30)),
12     new Quiz("年度健康問卷", LocalDate.of(2025, 4, 1), LocalDate.of(202
13  5, 4, 30))
14 );
15
16 LocalDate today = LocalDate.now();
17
18 List<Quiz> active = quizzes.stream()
19     .filter(q -> !today.isBefore(q.start()) && !today.isAfter(q.end
20     ))
21     .toList();
```



# 實戰：Stream groupingBy 統計問卷選項

後台統計頁需要知道每個選項各有幾票， `Collectors.groupingBy` + `Collectors.counting()` 一步到位：

```
1  import java.util.stream.*;
2
3  // 所有作答（單選題，從資料庫撈出後合併）
4  List<String> answers = List.of
5  ("A", "B", "A", "C", "A", "B", "C", "C", "B", "A");
6
7  // 一行計票，回傳 Map<選項, 票數>
8  Map<String, Long> tally = answers.stream()
9  .collect(Collectors.groupingBy(ans -> ans, Collectors.counting
10  (())));
11
12  System.out.println(tally); // {A=4, B=3, C=3}
13
14  // 依票數降序排列輸出（搭配長條圖 / 圓餅圖的資料準備）
15  tally.entrySet().stream()
16  .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
17  .forEach(e -> System.out.printf("選項 %s : %d 票\n", e.getKey(), e.g
18  etValue()));
```



## Q & A

# 課程結束

**掌握 Lambda 與 Stream，寫出更現代的 Java !**

如有課後疑問，歡迎來信討論。